

AI-Assisted Coding Lab Assignment - 13.4

Name: A. Sushmitha

Roll No: 2303A51541

Batch: 13

Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions

Task 1: Refactoring Data Transformation Logic

Scenario

You are maintaining a data preprocessing script where numerical transformations are written using verbose loops.

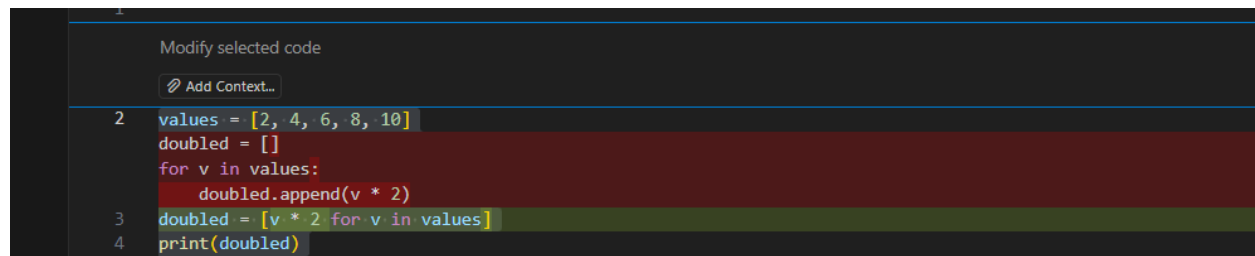
Legacy Code

```
values = [2, 4, 6, 8, 10]
doubled = []
for v in values:
    doubled.append(v * 2)
print(doubled)
```

Expected Output

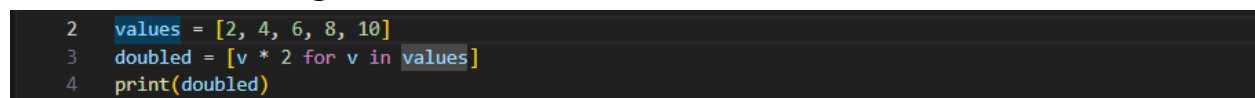
[4, 8, 12, 16, 20]

Code Before Refactoring:



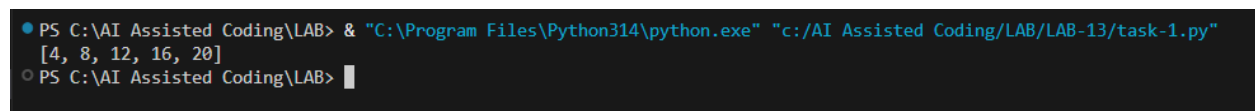
```
2 values = [2, 4, 6, 8, 10]
   doubled = []
   for v in values:
       doubled.append(v * 2)
3 doubled = [v * 2 for v in values]
4 print(doubled)
```

Code After Refactoring:



```
2 values = [2, 4, 6, 8, 10]
3 doubled = [v * 2 for v in values]
4 print(doubled)
```

Output:



```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:\AI Assisted Coding\LAB\LAB-13\task-1.py"
[4, 8, 12, 16, 20]
PS C:\AI Assisted Coding\LAB>
```

Explanation:

- The list comprehension `[v * 2 for v in values]` iterates through each element in values.
- Each value is multiplied by 2.
- The results are directly stored in the new list doubled.
- This preserves the same output as the original loop but in a more Pythonic way.

Task 2: Improving Text Processing Code Readability

Scenario

You are working on a log message generator that builds messages word by word.

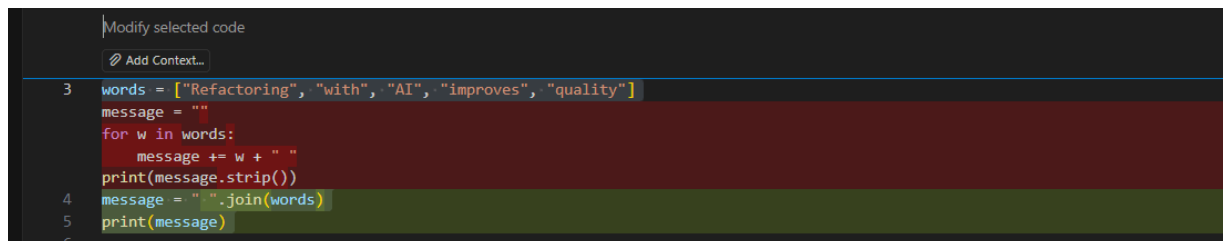
Legacy Code

```
words = ["Refactoring", "with", "AI", "improves",  
"quality"]  
message = ""  
for w in words:  
    message += w + " "  
print(message.strip())
```

Expected Output

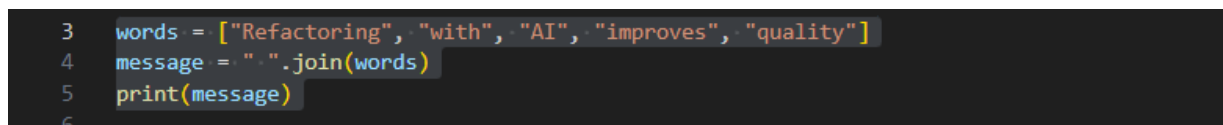
Refactoring with AI improves quality

Code Before Refactoring:



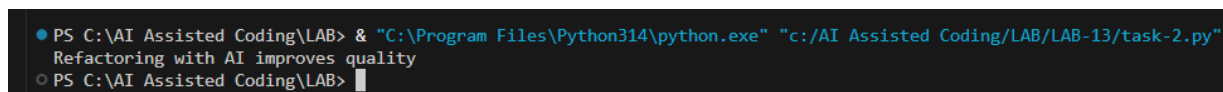
```
3 words = ["Refactoring", "with", "AI", "improves", "quality"]  
message = ""  
for w in words:  
    message += w + " "  
print(message.strip())  
4 message = " ".join(words)  
5 print(message)  
6
```

Code After Refactoring:



```
3 words = ["Refactoring", "with", "AI", "improves", "quality"]  
4 message = " ".join(words)  
5 print(message)  
6
```

Output:



```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-13/task-2.py"  
Refactoring with AI improves quality  
PS C:\AI Assisted Coding\LAB>
```

Explanation

- " ".join(words) combines all the words in the list into a single string.
- The " " (space) is placed between each word automatically.
- This method avoids repeated string concatenation and makes the code cleaner and faster while producing the same output.

Task 3: Safer Access to Configuration Data

Scenario

You are maintaining a configuration loader where dictionary keys may or may not exist depending on deployment settings.

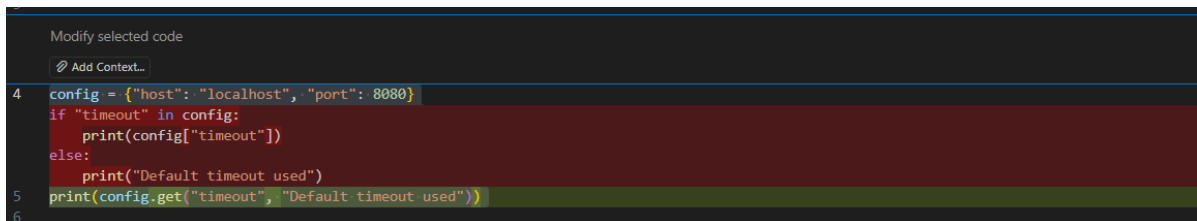
Legacy Code

```
config = {"host": "localhost", "port": 8080}
if "timeout" in config:
    print(config["timeout"])
else:
    print("Default timeout used")
```

Expected Output

Default timeout used

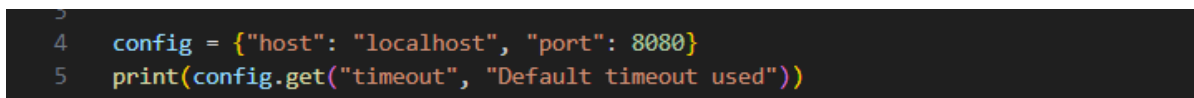
Code Before Refactoring:

A screenshot of a code editor with a dark theme. The code is as follows:

```
4 config = {"host": "localhost", "port": 8080}
5 if "timeout" in config:
6     print(config["timeout"])
7 else:
8     print("Default timeout used")
9 print(config.get("timeout", "Default timeout used"))
```

Line 4 is highlighted in blue, line 5 in red, line 6 in red, line 7 in red, line 8 in red, and line 9 in green. A toolbar at the top shows "Modify selected code" and "Add Context...".

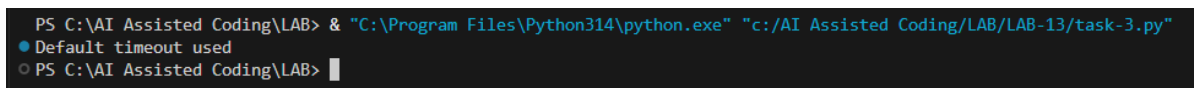
Code After Refactoring:

A screenshot of a code editor with a dark theme. The code is as follows:

```
4 config = {"host": "localhost", "port": 8080}
5 print(config.get("timeout", "Default timeout used"))
```

Line 4 is highlighted in blue and line 5 in green.

Output:

A screenshot of a Windows command prompt. The text shown is:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-13/task-3.py"
● Default timeout used
○ PS C:\AI Assisted Coding\LAB>
```

Explanation

- `config.get("timeout", "Default timeout used")` tries to retrieve the value for "timeout".
- If the key **exists**, it returns its value.
- If the key **does not exist**, it returns the **default message** "Default timeout used".
- This approach removes the need for an if-else check and makes the code **shorter, safer, and more readable**.

Task 4: Refactoring Conditional Logic for Scalability

Scenario

You are enhancing a utility module that performs different operations based on user input.

Legacy Code

```
action = "divide"
x, y = 10, 2
if action == "add":
    result = x + y
elif action == "subtract":
    result = x - y
elif action == "multiply":
    result = x * y
elif action == "divide":
    result = x / y
else:
    result = None
print(result)
```

Expected Output

5.0

Code Before Refactoring:

```
4 action = "divide"
5 x, y = 10, 2
6 if action == "add":
7     result = x + y
8 elif action == "subtract":
9     result = x - y
10 elif action == "multiply":
11     result = x * y
12 elif action == "divide":
13     result = x / y
14 else:
15     result = None
16
17 operations = {
18     "add": lambda a, b: a + b,
19     "subtract": lambda a, b: a - b,
20     "multiply": lambda a, b: a * b,
21     "divide": lambda a, b: a / b,
22 }
23
24 result = operations.get(action, lambda a, b: None)(x, y)
25 print(result)
```

Code After Refactoring:

```
3
4  action = "divide"
5  x, y = 10, 2
6
7  operations = {
8      "add": lambda a, b: a + b,
9      "subtract": lambda a, b: a - b,
10     "multiply": lambda a, b: a * b,
11     "divide": lambda a, b: a / b,
12 }
13
14 result = operations.get(action, lambda a, b: None)(x, y)
15 print(result)
```

Output:

```
PS C:\AI Assisted Coding\LAB> & "C:\Program Files\Python314\python.exe" "c:/AI Assisted Coding/LAB/LAB-13/task-4.py"
5.0
PS C:\AI Assisted Coding\LAB> █
```

Explanation

- A **dictionary (operations)** maps operation names to functions.
- `operations.get(action, lambda a, b: None)` retrieves the corresponding function.
- If the action is not found, it returns a default function that returns `None`.
- The selected function is then executed with `(x, y)`.
- This approach is **cleaner, easier to extend, and more scalable** than multiple if-elif statements.

Task 5: Simplifying Search Logic in Collections

Scenario

You are reviewing legacy inventory-check code that uses manual loops to find elements.

Legacy Code

```
inventory = ["pen", "notebook", "eraser", "marker"]
found = False
for item in inventory:
    if item == "eraser":
        found = True
        break
print("Item Available" if found else "Item Not Available")
```

Expected Output

Item Available

Code Before Refactoring:

```
Modify selected code
Add Context...

4 inventory = ["pen", "notebook", "eraser", "marker"]
5 found = False
6 for item in inventory:
7     if item == "eraser":
8         found = True
9         break
10 print("Item Available" if found else "Item Not Available")
11
12
13
14 # Refactored version using 'in' operator
15 found = "eraser" in inventory
16 print("Item Available" if found else "Item Not Available")
```

Code After Refactoring:

```
4 inventory = ["pen", "notebook", "eraser", "marker"]
5 found = False
6 for item in inventory:
7     if item == "eraser":
8         found = True
9         break
10 print("Item Available" if found else "Item Not Available")
11
12 # Refactored version using 'in' operator
13 found = "eraser" in inventory
14 print("Item Available" if found else "Item Not Available")
15
```

Output:

```
Item Available
PS C:\AI Assisted Coding\LAB>
```

Explanation

- The **in operator** directly checks whether "eraser" exists in the inventory list.
- It replaces the manual loop and flag variable (found).
- This makes the code **shorter, clearer, and easier to maintain** while keeping the same output.